

Scheduling jobs

So far, nothing has been discussed about how you can run a job (script) at a specified time. This is possible by means of a utility called **at**. When invoked with the option **-f**, it will run a given script at a given time. The following is a typical way you can use the **at** command:

```
at -f <script name> <time>
```

Here <time> can be specified in different ways, like:

- 11:45 PM
- now
- now+5 minutes

To get more details on **at**, check out the man page.

It is worth mentioning that the output of jobs scheduled using **at** will not be tagged with **STDOUT** (i.e., the monitor) which means that the process will run only in the background. We will be able to know whether the run was successful or not by checking the mailbox of the person who used **at**, or by using the system monitor utilities like **ps**.



Note: The mailbox here is different. In the current context, it's the e-mail sent by the system to the person (user) who has scheduled the job. If users have installed a utility called **mail**, the output will be written to a text file in the **/var/mail** directory. It is not compulsory that you need to have **mail** installed, but for those who are interested in scheduling more jobs using **at**, **mail** is a useful utility.

Let's suppose our schedule looks like the following:

- login and start download at 2.10 a.m.
- logout at 5.45 a.m.
- shutdown at 5.50 a.m.

What we need to do is schedule our scripts (**start.sh**, **logout.sh** and **shutdown.sh**) at the respective time. Open three terminal windows and run the following:

```
# at -f start.sh 2:10 AM ##in the first window
# at -f logout.sh 5:45 AM ##in the second window and
# at -f shutdown.sh 5:50 AM ## in the third window
```

Continuing from stopped downloads

If the download was not over within the time span, a truncated file will be created at the destination folder at the time of logout. This is true for large sized files. On those occasions, you can continue from where you've stopped at a convenient time, by running **wget** with the **-C** option:

```
wget -C <target folder> <URL>
```

If there is a file with the same name as the one specified within the URL in the target directory, **wget** will request the server to continue retrieval from an offset equal to the length of the downloaded one.

Improvisations that we can incorporate

Now, our scheduler can be made to run successfully under normal conditions, which of course includes an uninterrupted connection with the server. For those sites that compulsorily require you to sign in to access the Net, you can simply invoke **lynx-cur** by making it read keystrokes from a file (as we've done earlier) for signing in. In case of disturbances that result in disconnection while downloading, you can ensure a smooth download by making provisions in the script to reconnect. This can be done by writing a function as follows:

```
#download.sh
#!/bin/bash
function do_it {
    wget -t 1 --timeout=60 -c --directory-prefix=/media/new_volume/yesterday
}
status=57
count=${Count + 1}
if [ $status -ne 0 ] && [ $count -lt 5 ]
then
    bash login.sh
    do_it $1
fi
count=0

count=0
do_it <URL 1>
-----
do_it <URL n>
```

Here, **do_it** is a function which accepts URLs as arguments which **wget** is then made to point at. The **-t** option specifies number of tries **wget** must attempt to download. Its value is assigned as 1. After the first try, **wget** will quit with a status value. This is examined along with the value of an integer variable count to decide whether a re-login has to be made.

Integer variable count is specified by us, depending upon the number of times re-login must be allowed. Here, I gave the terminating count to be 5, which means download will continue smoothly by logging in for first five disconnections experienced in course of the download of a file. You can also see the **--timeout** option which can be used to specify the maximum time, in seconds, **wget** must wait for a reply from an idle server. Its default value is 900 seconds. It is set to 60 seconds just to save time.

I guess that's about it, for now. Will catch you later when I chance upon some other useful tip.

By: Sreekanth Balakrishnan

The author is a third-year B. Tech computer science student from College of Engineering (CET), Tiruvandrum. He is an active member of fsug-tvm and also the FOSS group at his college. He loves shell- and python-scripting, and is an evergreen fan of KDE.